A Review and Analysis of Software Complexity Metrics in Structural Testing

Mrinal Kanti Debbarma[1], Swapan Debbarma[2], Nikhil Debbarma[2], Kunal Chakma[2] and Anupam Jamatia[2]
[1]CSED, MNNIT, Allahabad, India,[2]CSED, NIT, Agartala, India
{mkdb06,swapanxavier,ndbarma.csenita,kchax4377, anupamjamatia}@gmail.com

***Abstract*—Software metrics is developed and used by the various software organizations for evaluating and assuring software code quality, operation, and maintenance. Software metrics measure various types of software complexity like size metrics, control flow metrics and data flow metrics. These software complexities must be continuously calculated, followed, and controlled. One of the main objectives of software metrics is that applies to a process and product metrics. It is always considered that high degree of complexity in a module is bad in comparison to a low degree of complexity in a module. Software metrics can be used in different phases of the software development lifecycle. This paper reviews the theory, called "software complexity metrics", and analysis has been done based on static analysis. We try to evaluate and analyze different aspects of software metrics in structural testing which offers of estimating the effort needed for testing.**

Key words: Software metrics, lines of code , Control flow metrics, NPATH complexity , Structural Testing.

I. INTRODUCTION

The Software complexity is based on well-known software metrics, this would be likely to reduce the time spent and cost estimation in the testing phase of the software development life cycle (SDLC), which can only be used after program coding is done. Improving quality of software is a quantitative measure of the quality of source code. This can be achieved through definition of metrics, values for which can be calculated by analyzing source code or program is coded. A number of software metrics widely used in the software industry are still not well understood [2]. Although some software complexity measures were proposed over thirty years ago and some others proposed later. Sometimes software growth is usually considered in terms of complexity of source code. Various metrics are used, which unable to compare approaches and results. In addition, it is not possible or equally easy to evaluate for a given source code [4]. Software complexity, deals with how difficult a program is to comprehend and work with [1]. Software maintainability [1], is the degree to which characteristics that hamper software maintenance are present and determined by software complexity. There dependencies are shown in Fig. 1.

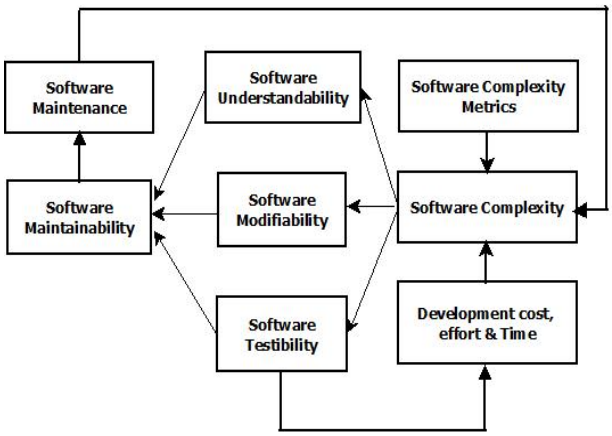

Fig. 1 Relationship between Software Complexity Metrics and Software Systems

This paper presents an analysis by which tester/developer can minimize software development cost and improve testing efficacy and quality.

II. PROBLEM STATEMENT:

From software engineering point of view software development experience shows, that it is difficult to set measurable targets when developing software products. Produced/developed software has to be testable, reliable and maintainable. On the other side, "You cannot control what you cannot measure" [3]. In software engineering field during software process, developers do not know if what they are developing is correct and guidance are needed to help them accustom more improvement. Software metrics are facilitating to track software enhancement. Various industries dedicated to develop software, and use software metrics in a regular basis. Some of them have produced their own standards of software measurement, so the use of software metrics is totally depending upon industry to industry. In this regards, what to measure is classified into two categories, such that software process or software product. But ultimately, main goal of this measure is customer satisfaction not only at delivery, but through the whole development process.

III. BACKGORUND AND RELATED WORK

(A) Software metrics:

Software metrics is defined by measuring of some property of a portion of software or its specifications. Software metrics provide quantitative methods for assessing the software quality. Software metrics can be define as: "The continuous application of measurement-based techniques to the software development process and its products to supply meaningful and timely management information (MI) together with the use of those techniques to improve its products and that process" [13].

(B) Software complexity:

Software complexity, deals with how difficult a program is to comprehend and work with [1]. Software maintainability [1], is the degree to which characteristics that hamper software maintenance are present and determined by software complexity. Software complexity is based on well known software metrics.

Various software complexity metrics invented and can be categorized into two types:

(I) Static metrics

Static metrics are obtainable at the early phases of software development life cycle (SDLC).

These metrics deals with the structural feature of the software system and easy to gather.

Static complexity metrics estimate the amount of effort needed to develop, and maintain the code.

(II) Dynamic metrics

Dynamic metrics are accessible at the late stage of the software development life cycle (SDLC). These metrics capture the dynamic behavior of the system and very hard to obtain and obtained from traces of code.

(C) Software Complexity Measures: Attributes

Software complexity metrics can be distinguished by the attributes used for measurement. In this paper, we are concentrating on static measure which can be classified into three types:

(i) Size based metrics:

Size is one of the most essential attributes of software systems [15]. It controls the expenditure incurred for the systems both in man-power and budget, for the development and maintenance. These metrics specify the complexity of software by size attributes and helps in predicting the cost involvement for maintaining the system. Size based metrics measures the actual size of the software module. These metrics is originated from the basic counts such as line numbers, volume, size, effort, length, etc.

(ii) Control flow based metrics:

Control flow based metrics measures the comprehensibility of control structures. These metrics also confine the relation between the logic structures in program with its program complexity. These metrics are originated from the control structure of a program [1].

(iii) Data flow based metrics:

Data flow based metrics measure the usage of data and their data dependency (visibility of data as well as their interactions) [1].

Summarized Software metrics are shown in Table1.

TABLE I. SOFTWARE METRICS: CLASSIFICATIONS

Type	Metrics	Description	Merit & De-merits
Size Metrics (Program Size)	Lines of Code (LOC), Token Counts (TC), Function Points (FP), Halstead's software science (HSS)	Metrics based on program size, amount of lines of code, declarations, statements, and files. Halstead's metrics are based on count of unique number of operators and operands in a program.	Easy to understand; fast to count, program language independent and widely applicable. No need of deep analysis of program's logic structure. In contrast ignores the complexity from the control flow.
Control flow based metrics (Program Control Structure)	McCabe's Cyclomatic Complexity (MCC), Conte's Average Nesting Level, (CANC), NPATH Complexity (NC)	Metrics based on control structure of the program or control flow graph (CFG) and density of control within the program Measure acyclic execution path through a program.	Ignores the complexity from the data flow of the program and Complexity added by the nesting levels. Do not distinguish the complexities of various kinds of control flow.
Data Flow based metrics	Chung's live definition	Metrics is based on use of data within a program.	Intra and inter module's data dependency complexity

Structural testing criteria consider on the knowledge of the internal structure of the program implementation to derive the testing criteria. Test cases are generated for actual implementation, if there is some change in implementation then it leads to change in test cases. They can be classified as, complexity, control flow and data flow based criteria. The complexity based criterion requires the execution of all independent paths of the program; it is based on McCabe's complexity concept [10]. For the control flow based criteria, testing requirements are based on the Control Flow Graph (CFG). It requires the execution of components (blocks) of the program under test in condition of subsequent elements of the CFG i.e. nodes, edges and paths. Another method is number of unit tests needed to test every combination of paths in a method. In Data Flow based criteria, both data flow and control flow information are used to perform testing requirements. These coverage criteria are based on code coverage. Code coverage is the degree to which source code of a program has been tested. Test coverage is measured during test execution. Once such a criterion has been selected, test data must be selected to fulfill the criterion.

Complexity of software is measuring of software code quality; it requires a model to convert internal quality attributes to code reliability. High degree of complexity in a component like function, subroutine, object, class etc. is consider bad in comparison to a low degree of complexity in a component. Software complexity measures which enables the tester to counts the acyclic execution paths through a component and improve software code quality. In a program characteristic that is one of the responsible factors that affect the developer's productivity [6] in program comprehension, maintenance, and testing phase. There are several methods to calculate complexity measures were investigated, e.g. different version of LOC [6], NPATH [7], McCabe's cyclomatic number [9], Data quality [9], Halstead's software science [8], Function Points[17], Token Counts[8], Nesting Level[15], Chung's live definition[16] etc.

IV. CLASIFICATION OF SOFTWARE METRICS:

Software metrics are useful to the software process, and product metrics. Various classification of software metrics are as follows:

i) Software Process metrics
ii) Software Product metrics

i). Software Process metrics:

Software process metrics involves measuring of properties of the development process and also known as management metrics. These metrics include the cost, effort, reuse, methodology, and advancement metrics. Also determine the size, time and number of errors found during testing phase of the SDLC.

ii). Software Product metrics:

Software process metrics involves measuring the properties of the software and also known as quality metrics. These metrics include the reliability, usability, functionality, performance, efficacy, portability, reusability, cost, size, complexity, and style metrics. These metrics measure the complexity of the software design, size or documentation created.

1.1.Size metrics : Lines of Code

The size of the program indicates the development complexity, which is known as Lines of Code (LOC). The simplest measure of software complexity recommended by Hatton(1977). This metric is very simple to use and measure the number of source instruction required to solve a problem. While counting a number of instructions (source), line used for blank and commenting lines are ignored. The size, complexity of today's software systems demands the application of effective testing techniques. Size attributes are used to describe physical magnitude, bulk etc. Lines of code and Halstead's software science [8] are examples of size metrics. M. Halstead proposed a metrics called software science.

1.2. Control Flow metrics: NPATH Complexity [7]

The control flow complexity metrics are derived from the control structure of a program. The control flow measure by NPATH, invented by Nejmeh [7] ,it measures the acyclic execution paths, NPATH is a metric which counts the number of execution path through a functions. NPATH is an example of control flow metrics.

One of the popular software complexity measures NPATH complexity (NC) is determined as:

$$\text{NPATH}=\prod_{i=1}^{i=N} NP(\text{statement}_i)$$
$$\text{NP(if)}=\text{NP(expr)}+\text{NP(if-range)}+1$$
$$\text{NP(if-else)}=\text{NP(expr)}+\text{NP(if-range)}+\text{NP(else-range)}$$
$$\text{NP(while)}=\text{NP(expr)}+\text{NP(while-range)}+1$$
$$\text{NP(do-while)}=\text{NP(expr)}+\text{NP(do-range)}+1$$
$$\text{NP(for)}=\text{NP(for-range)}+\text{NP(expr1)}+\text{NP(expr2)}+\text{NP(expr3)}+1$$
$$\text{NP("?")}=\text{NP(expr1}+\text{NP(expr2)}+\text{NP(expr3)}+2$$
$$\text{NP(repeat)}=\text{NP(repeat-range)}+1$$
$$\text{NP(switch)}=\text{NP(expr)}+\sum_{i=1}^{i=n} NP(case-range)+\text{NP(default-range)}$$
$$\text{NP(function call)}=1$$
$$\text{NP(sequential)}=1$$
$$\text{NP(return)}=1$$
$$\text{NP(continue)}=1$$
$$\text{NP(break)}=1$$
$$\text{NP(goto label)}=1$$
$$\text{NP(expressions)}=\text{Number of \&\& and || operators in Expression}$$

Execution of Path Expressions (complexity expression) are expressed, where "N" represents the number of statements in the body of component (function and "NP (Statement)" represents the acyclic execution path complexity of statement i. where "(expr)" represents expression which is derived from flow-graph representation of the statement. For example NPATH measure as follows:

```
Void func-if-else ( int c)
{
  int a=0;
  if(c)
   {
    a=1;
   }
  else
   {
  a=2;
  }
 }
```

The Value of NPATH = 2 as follows:

$$\text{NP (if-else)}=\text{NP(expr)}+\text{NP(if-range)}+\text{NP(else-range)}$$

In the above example, NP (exp)=0 for if statement. NP (If-range)=1 for if statement and , NP(else-range)=1 for if-else statement. So, NP (if-else)=0+1+1=2.

NPATH, metric of software complexity overcomes the shortfalls of McCabe's metric which fail to differentiate between various kinds of control flow and nesting levels control structures.

1.3. McCabb'e Cyclomatic Complexity [9]

Cyclomatic Number is one of the metric based on not program size but more on information/control flow. It is based on specification flow graph representation developed by Thomas J Mc Cabb in 1976. Program graph is used to depict control flow. Nodes are representing processing task (one or more code statement) and edges represent control flow between nodes. McCabe's metrics [10] is example of control flow metrics. To compute Cyclomatic Number by V (G) as following methods:

$$V(G)=E-N+2P$$

Where, V (G)= Cyclomatic Complexity
E= the number of edges in a graph
N= the number of nodes in graph
P= the number of connected components in graph,
We can compute the number of binary node (predicate), by the following equation.

$$V(G)=p+1$$

Where, V(G)= Cyclomatic Complexity
P= number of nodes or predicates

The problem with McCabb's Complexity is that, it fails to distinguish between different conditional statements (control flow structures). Also does not consider nesting level of various control flow structures. NPATH, have advantages over the McCabb's metric [7].

1.4. Halstead Software Science Complexity[8]

M. Halstead's [8] introduced software science measures for software complexity product metrics. Halseatd's software science is based on a enhancement of measuring program size by counting lines of code. Halstead's metrics measure the number of number of operators and the number of operands and their respective occurrence in the program (code). These operators and operands are to be considered during calculation of Program Length, Vocabulary, Volume, Potential Volume, Estimated Program Length, Difficulty, and Effort and time by using following formulae.

n_1 = number of unique operators,
n_2 = number of unique operands,
N_1 = total number of operators, and
N_2 = total number of operands,

a) Program Length (N) = N1+N2
b) Program Vocabulary (n) = n1+n2
c) Volume of a Program (V) = $N*\log_2 n$
d) Potential Volume of a Program (V*)=$(2+n_2)\log_2(2+n_2)$
e) Program Level (L) = L=V*/V
f) Program Difficulty (D) = 1/L
g) Estimated Program Length (N) = $n_1\log_2 n_1+n_2\log_2 n_2$
h) Estimated Program Level (L) = $2n_2/(n_1N2)$
i) Estimated Difficulty (D) = $1/L = n_1N_2/2n_2$
j) Effort (E) = $V/L = V*D = (n_1 \times N_2) / 2n_2$
k) Time (T) = E/S ["S" is Stroud number (given by John Stroud), the constant "S" represents the speed of a programmer. The value "S" is 18]

One major weakness of this complexity is that they do not measure control flow complexity and difficult to compute during fast and easy computation.

V. STATIC ANANYSIS

Our analysis is based on static analysis of software complexity metrics like size and control flow metrics. We have considered four program characteristics from the literature that are responsible for complexity measures. e.g., LOC, NC, MCC, and HSSC. For this study, we have selected only program written in C language given in Fig.2. We have measured LOC, NPATH i.e. acyclic execution paths through components for in an attempt at program optimization, McCabe complexity and finally Halstead's software science complexity metrics. Statics analysis of metrics is not directly associated to the execution of programs (source code). There are three aspects can be affect maintenance of program, like program volume/size, data organization, and control structure.

While counting a number of instructions (source), line used for blank and commenting lines are ignored. NPATH measures the acyclic execution paths which counts the number of execution path through a functions. Halstead's metrics measure the number of number of operators and the number of operands and their respective occurrence in the program (code). These operators and operands are to be considered during calculation of Program Length, Vocabulary, Volume, Potential Volume, Estimated Program Length, Difficulty, and Effort and time. For McCabb's complexity measures program graph is used to depict control flow. Nodes are representing processing task (one or more code statement) and edges represent control flow between nodes

Consider an example, Let P be the source program in C given below:

Consider a program from fig.2, the complexity measured by us and computed the complexity of the other proposed measures i.e Lines of Code (LOC), NPATH Complexity (NC), McCabb's complexity (MCC) and Halstead's software science complexity (HSSC) are shown in Table II.

```
    #include<stdio.h>
    void main()
1:  {
1:   int a,b,c,n;
1:   scanf("%d %d", &a, &b);
2:   if (a < b)
2:   {
3:   c = a;
3:   }
4:   if(c = = b)
```

```
4:   {
5:   c= =a+1;
5:   }
5:   }
5:   else
5:   {
6:   c = b;
7:   if(c = =b)
7:   {
8:   c = b+1;
8:   }
8:   }
9:   n = c;
10:  while ( n <= 8 )
10:  {
11:  if ( b > c )
11:  {
12:  c = 2;
12:  }
12:  else
12:  {
13:  n = n + c +7;
14:  }
14:  n = n + 1;
14:  }
15:  Printf("%d%d%d",a,b,n);
15:  }
```

Fig. 2. Source Program P

TABLE II. CALCULATION OF THE COMPLEXITY MEASURES FROM PROGRAM P

LOC	NC	MCC		HSSC	
35	24	Nodes	15	n1	15
				n2	8
		Edges	19	N1	56
				N2	40
		Pred. Nodes	5	Vocabu Lary	434
				Estt. Program Length	82.6
		Regions	5	Difficulty	37.5
				Effort	16275
		V(G)	6	Time	904.16

VI. CONCLUSION

Software complexity metrics have a tendency to be used in judging the quality of software development and one of the vital parts of the SDLC. The volume, control and data based complexity are importance of today's software systems demand the application of effective testing techniques. In addition, it was observed that software complexity metrics which enables the tester to counts the acyclic execution paths through a program and improve software quality. This static analysis could be lead to reduce software development cost and improve testing efficacy and software quality by evaluating software complexity metrics with LOC, NPATH (NC), McCabb's complexity metrics (MCC) and Halstead's Software Science Complexity (HSSC).

REFERENCES

[1] W.Harrison, K Magel, R. Kluczny and A. Dekok, " Applying software complexity metrics to program maintenance," Compute, Vol. 15 pp-.65-79,1982.

[2] Thomas J.McCabe "A Complexity Measure" ,IEEETransactions on Software Engineering, vol, se-2,1976

[3] T.DeMarco; Controlling Software Projects; Prrntice Hall,New York,1982.

[4] Israel Herraiz, Jesus M. Gonzalez-Barahona, Gregorio Robles," Towards a theoretical model for software growth" in 29th International Conference on Software Engineering Workshops(ICSEW'07).

[5] B. Beizer, Software Testing Techniques, Van Nostrand Reinhold, 2nd Ed.,1990.

[6] S. D. Conte, H.E Dunsmore, and V.Y. Shen. Software Engineering Metrics and Models. Benjamin/Cummings Publishing Company, Inc., 1986.

[7] B.A. Nejmeh. NPATH: A Measure of Execution Path Complexity and Its Applications. Comm. of the ACM, 31(2):188-210, February 1988.

[8] M. Halstead. Elements of Software Science. NorthHolland,1977.

[9] T.A. McCabe. A Complexity Measure. IEEE Transactions on Software Engineering, 2(4):308-320, December 1976

[10] A.Fitzsimmons and T. Love;" A Review and Evaluation of Software Science",Computing Survey,Vol.10,No.1 ,March 1978

[11] Norman Fenton, "Software Measurement: A Necessary Scientific Basis"IEEE Transaction on Software Engineering, Vol.20,No.3, March,1994,

[12] Software Engineering – A Practitioner's Approach, Roger S. Pressman; McGraw-Hill International Edition.

[13] Goodman, Paul: "Practical Implementation of Software Metrics", London, McGraw Hill, 1993.

[14] J. Verner and G. Tate " A software size model", IEEE Transaction on software Engineering , vol 18, No.4, 1992.

[15] W. Harrision and L.I Magel, "A Complexity Based on Nesting Level", SIGPLAN Notice, 16(3), 1981.

[16] C.M Chung and M.G Yang " A Software Maintainability Measurement", Proceedings of the 1988 Science, Engineering & Tech. Houston, Texas, pp.V12-16.

[17] Everald E. Millis " Software Metrics", SEI Curriculam Module SEI- CM-12-2.1, Dec, 1988.